

DevOps – Despliegue de Aplicación en Kubernetes Local

Proyecto grupal – Integrantes:

- Matias Brunaga
- Andrés Ibarra
- Lian Yang
- Santiago Zarza

Repositorio de GitHub: <https://github.com/Andoumeda/devops-k8s>

Primero se deben instalar los paquetes necesarios:

```
sudo apt install kubectl kubecolor kind
```

Luego se debe crear un clúster en Kind:

```
kind create cluster
```

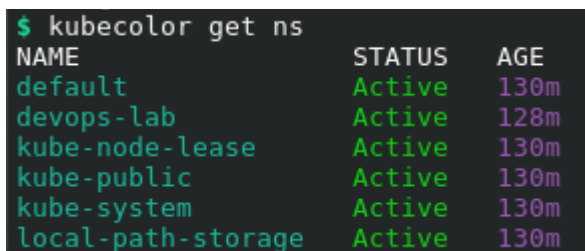
Comando para ver los namespaces en el clúster:

```
kubecolor get ns
```

Ahora se crea el namespace definido en el manifiesto:

```
kubecolor apply -f namespace.yml
```

La lista de namespaces debe verse así:



```
$ kubecolor get ns
```

NAME	STATUS	AGE
default	Active	130m
devops-lab	Active	128m
kube-node-lease	Active	130m
kube-public	Active	130m
kube-system	Active	130m
local-path-storage	Active	130m

Se construyen los contenedores Docker para frontend y backend:

```
docker build -t todo-frontend ./frontend/
```

```
docker build -t todo-backend ./backend/
```

Se cargan dichos contenedores en el clúster de Kind:

```
kind load docker-image todo-frontend
```

```
kind load docker-image todo-backend
```

Se crean los ConfigMaps, Deployments, Services y Secrets necesarios y definidos en los manifiestos del directorio:

```
kubecolor apply -f .
```

Ahora se obtienen los pods y servicios creados para verificar que están corriendo:

```
kubecolor get pods -n devops-lab
```

```
kubecolor get svc -n devops-lab
```

```
$ kubecolor get pods -n devops-lab
```

NAME	READY	STATUS	RESTARTS	AGE
backend-79969bbf6d-zvqbs	1/1	Running	0	87m
db-68c89bd9b5-pq2md	1/1	Running	0	23m
frontend-54f8c4f89b-bx9x7	1/1	Running	0	86m

```
$ kubecolor get svc -n devops-lab
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend-service	ClusterIP	10.96.136.248	<none>	3000/TCP	87m
db-service	ClusterIP	10.96.147.114	<none>	5432/TCP	87m
frontend-service	NodePort	10.96.14.42	<none>	80:30007/TCP	86m

Se activa port forwarding para el servicio de frontend y backend:

```
kubecolor port-forward svc/frontend-service 8080:80 -n devops-lab
```

```
kubecolor port-forward svc/backend-service 3000:3000 -n devops-lab
```

```
$ kubecolor port-forward svc/frontend-service 8080:80 -n devops-lab
```

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

```
$ kubecolor port-forward svc/backend-service 3000:3000 -n devops-lab
```

```
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
```

Al escalar el Deployment de backend a 3 réplicas, se puede ver que existen 3 pods asignados a backend:

```
$ kubecolor get deploy -n devops-lab
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
backend	1/1	1	1	154m
db	1/1	1	1	154m
frontend	1/1	1	1	154m

```
$ kubectl scale deployment backend --replicas=3 -n devops-lab
deployment.apps/backend scaled
andro@mx:~/Documents/k8s/manifests
$ kubectl get deploy -n devops-lab
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
backend	3/3	3	3	158m
db	1/1	1	1	158m
frontend	1/1	1	1	157m

```
$ kubectl get pods -n devops-lab
```

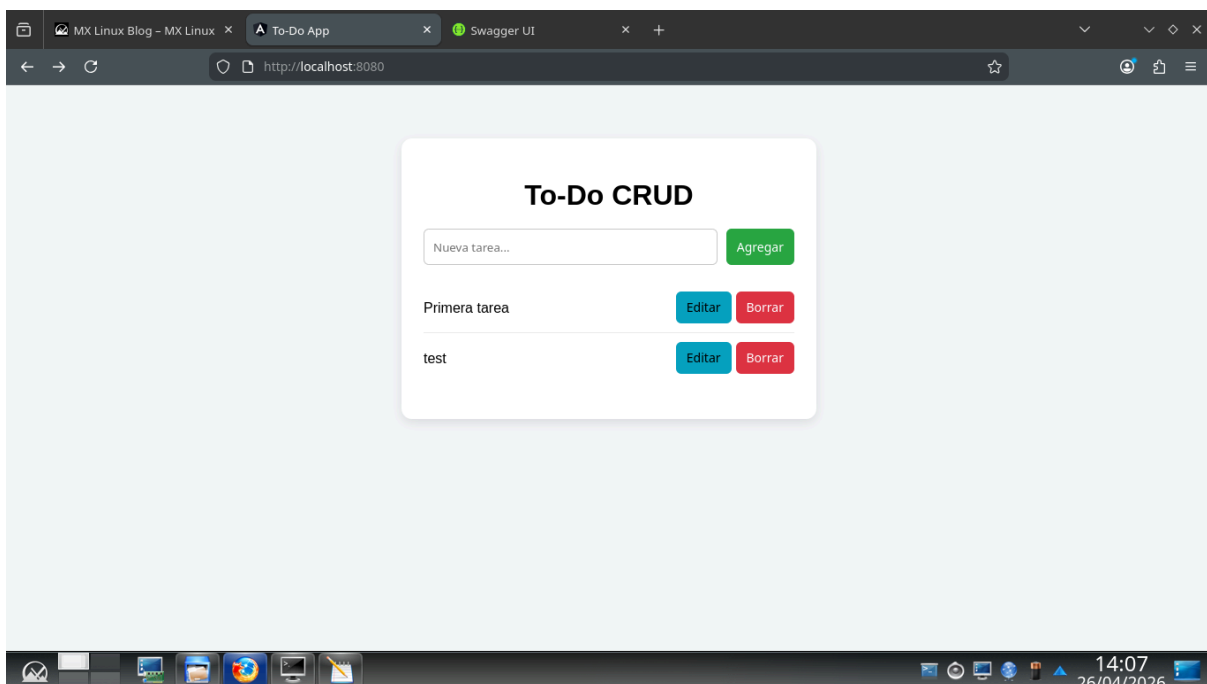
NAME	READY	STATUS	RESTARTS	AGE
backend-79969bbf6d-25vlh	1/1	Running	0	6m27s
backend-79969bbf6d-g7gsx	1/1	Running	0	6m27s
backend-79969bbf6d-zvqbs	1/1	Running	0	164m
db-68c89bd9b5-pq2md	1/1	Running	0	100m
frontend-54f8c4f89b-bx9x7	1/1	Running	0	164m

Al borrar un pod y luego volver a ver la lista de pods se ve que Kubernetes creó otro pod automáticamente para completar las 3 réplicas

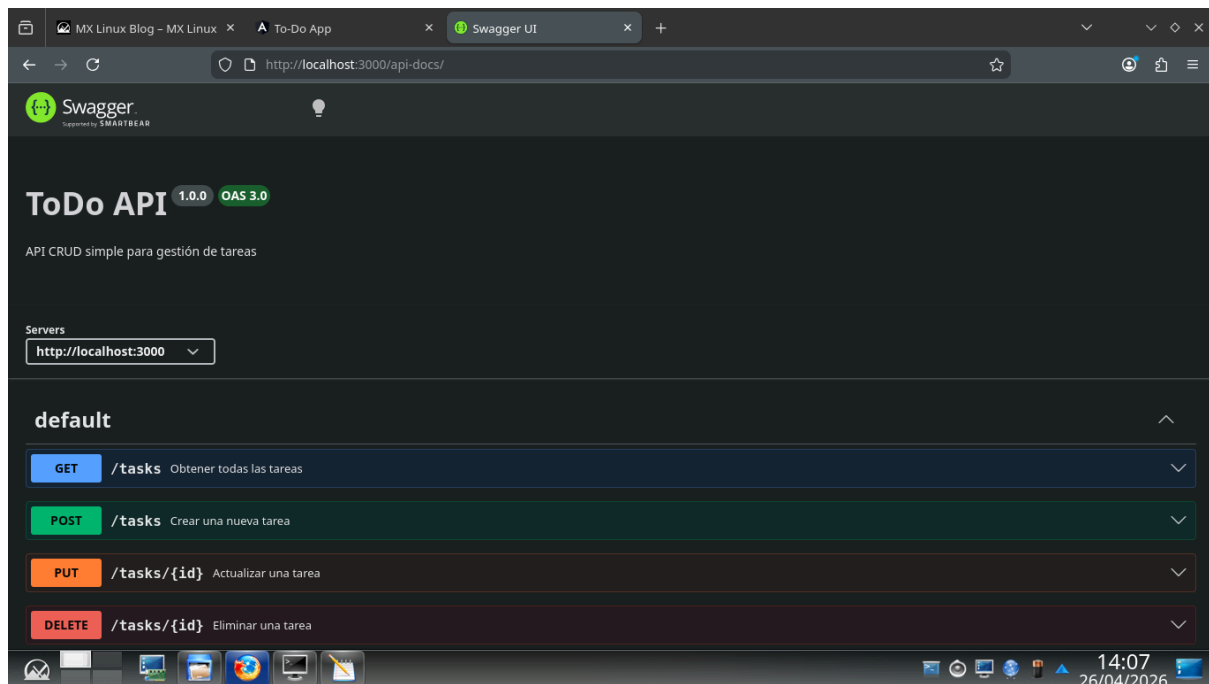
```
$ kubectl delete pod -n devops-lab backend-79969bbf6d-zvqbs
pod "backend-79969bbf6d-zvqbs" deleted
andro@mx:~/Documents/k8s/manifests
$ kubectl get pods -n devops-lab
```

NAME	READY	STATUS	RESTARTS	AGE
backend-79969bbf6d-25vlh	1/1	Running	0	8m35s
backend-79969bbf6d-g7gsx	1/1	Running	0	8m35s
backend-79969bbf6d-r8jnz	1/1	Running	0	40s
db-68c89bd9b5-pq2md	1/1	Running	0	102m
frontend-54f8c4f89b-bx9x7	1/1	Running	0	166m

Frontend ejecutando:



API Swagger de Backend funcionando:



Preguntas

1. ¿Cuál es la diferencia entre un Pod y un Deployment?

Un Pod es la unidad mínima en Kubernetes, puede tener 1 o más contenedores; mientras que un Deployment permite varios pods y sus réplicas, y trata de mantenerlos activos.

2. ¿Por qué Kubernetes recrea un pod automáticamente?

Sucede cuando el Pod está dentro de un Deployment, porque éste trata de mantener activo el número de réplicas que fue configurado.

3. ¿Qué problema resuelve un Service?

Un Service puede proporcionar una IP fija y balanceo de carga para poder comunicarse con los Pods desde el exterior.

4. ¿Cuál es la diferencia entre ConfigMap y Secret?

ConfigMap sirve para configuraciones públicas, mientras que Secret almacena configuraciones sensibles como contraseñas o claves API.

5. ¿Qué ocurre si un pod falla constantemente?

Kubernetes lo reinicia o intenta levantarlo varias veces en bucle, y si sigue sin funcionar entra en estado CrashLoopBackOff.

6. ¿Cómo se logra la escalabilidad en Kubernetes?

Puede ser manual, definiendo el número de réplicas en un Deployment; o automático, usando algún Autoscaler.

7. ¿Qué ventajas ofrece Kubernetes frente a Docker standalone?

Réplicas o copias de cada contenedor, facilidad de configuración para diversos nodos, escalabilidad y recuperación ante errores.

8. ¿Qué dificultades encontraron durante el despliegue?

Hubo problemas cuando, por ejemplo, Kubernetes no reconocía las imágenes Docker locales, o que el frontend no se comunicaba con el backend; y para solucionarlos había que cargar en el clúster Kind las imágenes Docker, y había que hacer port forwarding en los servicios de frontend y backend, respectivamente.

9. ¿Tuvieron errores y cómo diagnosticaron los errores?

*Al intentar usar `kubectl` sin definir antes un cluster en Kind, daba un error **E0423***

*Al intentar levantar un deployment sin cargar su imagen Docker local, daba el error **ImagePullBackOff***

*Al intentar escalar un deployment sin definir el namespace, daba los errores **No objects passed to scale** y **deployments.apps not found***

Estos errores fueron diagnosticados viendo la documentación de K8s y preguntando a la IA.

10. ¿Qué mejoras implementarían en producción?

Usar un proveedor externo de Secrets o una configuración con encriptación para que los datos sensibles estén más protegidos, además de un balanceador de carga en caso de recibir muchas peticiones, Ingress para facilitar el acceso externo sin port forwarding, y "requests and limits" para limitar los recursos que pueden usarse.